

PathForge V2 — Human Review: Decisions & Remarks

Completed review of every family in the simplified V2 review sheet. Judged solely on algorithmic approach vs. proposed label, per the review guide.

Summary

APPROVED: 16 | CORRECTED LABEL: 20 | REJECTED: 1

LC 1 — Two Sum

Family	Proposed Label	Decision	Corrected Label	Remark / Reason
lc1_fam1 (3 sol)	hash_lookup	APPROVED	—	Dictionary stores values and looks up the complement; the label matches exactly.
lc1_fam2 (2 sol)	No label	CORRECTED LABEL	brute_force	Nested for-loops checking every pair is brute force, not hash lookup.
lc1_fam3 (1 sol)	No label	CORRECTED LABEL	brute_force	Nested while-loops — same brute-force idea as fam2, different syntax.
lc1_fam4 (2 sol)	No label	CORRECTED LABEL	two_pointers_opposite	After sorting, low/high indices move inward from opposite ends — classic two-pointer, not hash-based.

LC 21 — Merge Two Sorted Lists

Family	Proposed Label	Decision	Corrected Label	Remark / Reason
lc21_fam1 (1 sol)	forward_pointer_advance	APPROVED	—	Iteratively advances pointers, always taking the smaller node — matches the label.
lc21_fam2 (2 sol)	No label	CORRECTED LABEL	sort_and_rebuild	Collects all values, sorts them, then rebuilds a list — not pointer merging at all.
lc21_fam3 (2 sol)	forward_pointer_advance	APPROVED	—	Dummy-node merge is the same forward-pointer-advance idea as fam1, just with a sentinel head.
lc21_fam4 (3 sol)	No label	CORRECTED LABEL	recursive_merge	Recursion, not iteration — forward_pointer_advance implies iterative pointer movement; this is a recursive merge instead.

LC 46 — Permutations

Family	Proposed Label	Decision	Corrected Label	Remark / Reason
lc46_fam1 (3 sol)	No label	CORRECTED LABEL	iterative_insertion	Builds permutations by inserting each element into every position of existing permutations — no recursion, so not dfs backtracking.
lc46_fam2 (2 sol)	dfs_backtracking	APPROVED	—	Recursive path-building with skip-if-used logic is textbook backtracking.
lc46_fam3 (3 sol)	dfs_backtracking	APPROVED	—	Recursive backtracking with an explicit used[] array and state restoration — matches the label.

LC 70 — Climbing Stairs

Family	Proposed Label	Decision	Corrected Label	Remark / Reason
lc70_fam1 (3 sol)	dp_bottom_up	APPROVED	—	Classic bottom-up DP array building from smaller to larger states.
lc70_fam2 (3 sol)	No label	CORRECTED LABEL	dp_top_down	Recursion with memoization is top-down DP, the opposite direction from dp_bottom_up.
lc70_fam3 (2 sol)	No label	CORRECTED LABEL	dp_bottom_up	Two rolling variables are a space-optimized version of the same bottom-up

Family	Proposed Label	Decision	Corrected Label	Remark / Reason
				recurrence — still dp_bottom_up in essence.

LC 102 — Binary Tree Level Order Traversal

Family	Proposed Label	Decision	Corrected Label	Remark / Reason
lc102_fam1 (4 sol)	bfs_shortest_path	APPROVED	—	Queue-based level-by-level traversal matches the vocabulary definition of bfs_shortest_path (queue-driven level exploration).
lc102_fam2 (4 sol)	No label	CORRECTED LABEL	recursive_dfs_by_depth	Recursive DFS that records values by depth — a different traversal strategy from the BFS family above.

LC 125 — Valid Palindrome

Family	Proposed Label	Decision	Corrected Label	Remark / Reason
lc125_fam1 (3 sol)	two_pointers_opposite	APPROVED	—	Two indices at both ends, skipping invalid characters and moving inward — matches the label exactly.
lc125_fam2 (2 sol)	No label	CORRECTED LABEL	clean_and_compare_reverse	Cleans the string then compares it to its reverse — no index pointers involved, so not two_pointers_opposite.
lc125_fam3 (3 sol)	No label	CORRECTED LABEL	two_pointers_opposite (recursive)	Still comparing left/right positions moving inward, just implemented recursively instead of with a while-loop.

LC 209 — Minimum Size Subarray Sum

Family	Proposed Label	Decision	Corrected Label	Remark / Reason
lc209_fam1 (4 sol)	sequential_accumulation + sliding_window	APPROVED	—	Expand-then-shrink variable window with a running sum matches both parts of the compound label.
lc209_fam2 (2 sol)	No label	CORRECTED LABEL	brute_force	Nested loops recomputing each subarray sum — no window pointers, so it's brute force, not sliding_window.
lc209_fam3 (2 sol)	No label	CORRECTED LABEL	sequential_accumulation	Builds a prefix-sum array (sequential accumulation) but checks it with nested loops, not a moving window — sliding_window would be inaccurate.

LC 242 — Valid Anagram

Family	Proposed Label	Decision	Corrected Label	Remark / Reason
lc242_fam1 (3 sol)	frequency_counting	APPROVED	—	Fixed-size count array incrementing/decrementing character frequencies matches the label.
lc242_fam2 (2 sol)	No label	REJECTED	—	The sheet itself flags zero-evidence members alongside a Counter-equality member; family coherence can't be confirmed from what's given, so it shouldn't be reviewed as one approved family.
lc242_fam3 (3 sol)	frequency_counting	APPROVED	—	Manual dictionary counting with decrement-and-check is frequency counting, just without the fixed-size array.

LC 547 — Number of Provinces

Family	Proposed Label	Decision	Corrected Label	Remark / Reason
lc547_fam1 (2 sol)	No label	CORRECTED LABEL	bfs_shortest_path	Queue-based traversal of the adjacency matrix with a visited array — matches the BFS vocabulary definition.
lc547_fam2 (3 sol)	No label	CORRECTED LABEL	recursive_dfs_traversal	Recursive DFS marking visited nodes — no choice-and-restore backtracking, so

Family	Proposed Label	Decision	Corrected Label	Remark / Reason
				dfs_backtracking would overstate it; plain recursive DFS traversal fits better.
lc547_fam3 (3 sol)	union_find	APPROVED	—	Parent array with find/union operations is exactly Union-Find.

LC 560 — Subarray Sum Equals K

Family	Proposed Label	Decision	Corrected Label	Remark / Reason
lc560_fam1 (2 sol)	No label	CORRECTED LABEL	brute_force	Nested loops recomputing every subarray total — no hashing or prefix structure involved.
lc560_fam2 (2 sol)	frequency_counting + sequential_accumulation	APPROVED	—	Running prefix sum plus a hash-map of prefix-sum frequencies matches both parts of the label.
lc560_fam3 (2 sol)	No label	CORRECTED LABEL	frequency_counting + sequential_accumulation	Same core algorithm as fam2 (prefix sum + dict frequency), just written with .get()/assignment instead of defaultdict.
lc560_fam4 (2 sol)	No label	CORRECTED LABEL	sequential_accumulation	Explicit prefix-sum array checked with nested loops, not a frequency map — frequency_counting would be inaccurate here.

LC 704 — Binary Search

Family	Proposed Label	Decision	Corrected Label	Remark / Reason
lc704_fam1 (4 sol)	binary_search	APPROVED	—	Standard iterative lo/hi/mid interval narrowing — matches the label exactly.
lc704_fam2 (1 sol)	No label	CORRECTED LABEL	binary_search (library-based)	bisect_left performs binary search internally; still fundamentally binary_search, just delegated to the standard library rather than hand-written.
lc704_fam3 (3 sol)	No label	CORRECTED LABEL	binary_search (recursive)	Recursive halving of the search interval is still binary_search, just implemented via recursion instead of a loop.

LC 3236 — Smallest Missing Integer Greater Than Sequential Prefix Sum

Family	Proposed Label	Decision	Corrected Label	Remark / Reason
lc3236_fam1 (7 sol)	sequential_accumulation	APPROVED	—	Running sum over the leading consecutive sequence is exactly sequential accumulation.
lc3236_fam2 (1 sol)	sequential_accumulation	APPROVED	—	Prefix array over the same leading sequence is the same accumulation idea, just materialized as an array.

Notes on labels used beyond the quick-vocabulary list

The review guide explicitly allows "brute_force" as a corrected label (its own LC1 fam2/fam3 example). A few other families needed a descriptive label not in the fixed vocabulary list (e.g. sort_and_rebuild, recursive_merge, iterative_insertion, dp_top_down, recursive_dfs_by_depth, clean_and_compare_reverse, recursive_dfs_traversal). These follow the same plain-English, approach-describing style as the provided vocabulary and are flagged above for your discretion — swap in your own team's preferred term if you have a house-style equivalent.

lc242_fam2 flagged REJECTED

The sheet's own description for this family says it "contains zero-evidence members" alongside one Counter-equality member. Since coherence of the family (whether all members share one approach) can't be verified from what's given, this is the one case matching the REJECTED criterion — not because the visible code is wrong, but because the family's coherence is unconfirmed.